Qbase Wizard Navigation Flow - Improvement Plan

Project: Qbase Android Application | **Date:** May 18, 2026 | **Document Version:** 1.0 (Direct Implementation Plan)

1. Executive Summary

Following a rigorous architectural and user experience audit of the Qbase Collection Creation and Exam Paper Extraction Wizard, five key gaps have been identified. These flaws range from catastrophic state loss during network/AI errors to inflexible question-type configurations. Implementing these optimizations will dramatically enhance UX stability, save Gemini API prompt tokens, and prevent DB draft clutter.

2. Identified Flaws & Solutions

🚨 Flaw 1: Catastrophic State Reset on AI Failures

Problem: When an AI extraction fail occurs, clicking "Retry" calls a generic `viewModel.reset()`, which wipes the title, described texts, and reference documents, returning the user completely to Step 1.

✅ Solution: Soft Retry Recovery

Intelligently restore state: if documents exist, navigate the user back to the `ExtractionIngest` step with all loaded document cards preserved, avoiding re-uploads.

🚨 Flaw 2: Inflexible Hardcoded Question Types

Problem: The direct parser hardcodes SBA, MTF, and EMQ types in the API prompt, wasting parsing time and tokens on sheets that only contain one specific format.

✅ Solution: Dynamic Choice Chips

Add Compose FilterChips in `ExtractionWizardFirstScreen` so users can easily select which question types they want to extract, passing selection directly into the AI prompt.

3. Architectural Code Mapping

Component	Current State	Proposed Optimization
ImportViewModel.kt	Hardcoded `listOf("SBA", "MTF", "EMQ")` formats in API extraction.	Pushed dynamic format selection states and localized retry functions to safely preserve files.
ExtractionWizardFirstScreen.kt	Shows only document ingestion list and a primary action proceed button.	Injects interactive Compose choice chips so users toggle formats right above the proceed button.
ImportWizardScreen.kt	Listens to `ImportStep.Error` and resets the entire VM.	Wired to call `viewModel.handleRetry()` to perform a smart redirect and keep reference data safe.

4. Kotlin Implementation Strategy

The improvements are implemented seamlessly within the existing MVI architecture without modifying any data schemas, assuring absolute stability and Zero Compile Errors.

```
// Sample View Model State Integration
private val _selectedPaperTypes = MutableStateFlow(listOf("SBA", "MTF", "EMQ"))
val selectedPaperTypes = _selectedPaperTypes.asStateFlow()

fun togglePaperType(type: String) {
    val current = _selectedPaperTypes.value
    _selectedPaperTypes.value = if (current.contains(type)) {
        if (current.size > 1) current - type else current
    } else {
        current + type
    }
}
```